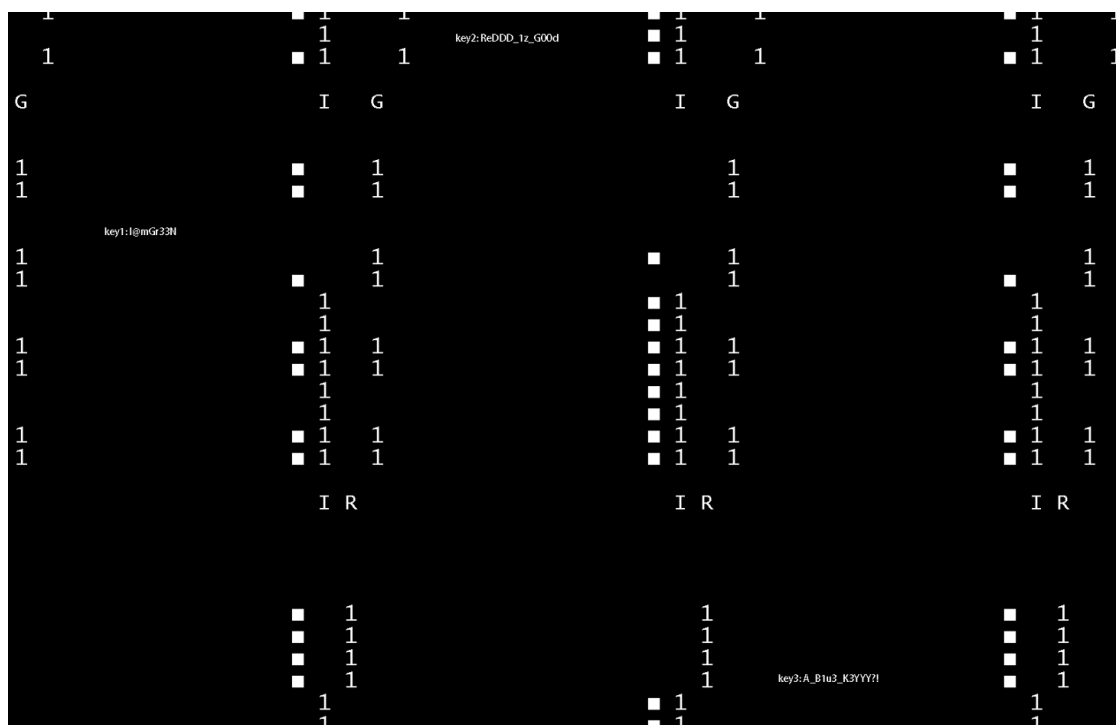


ISCC2025 WriteUp 提交模板

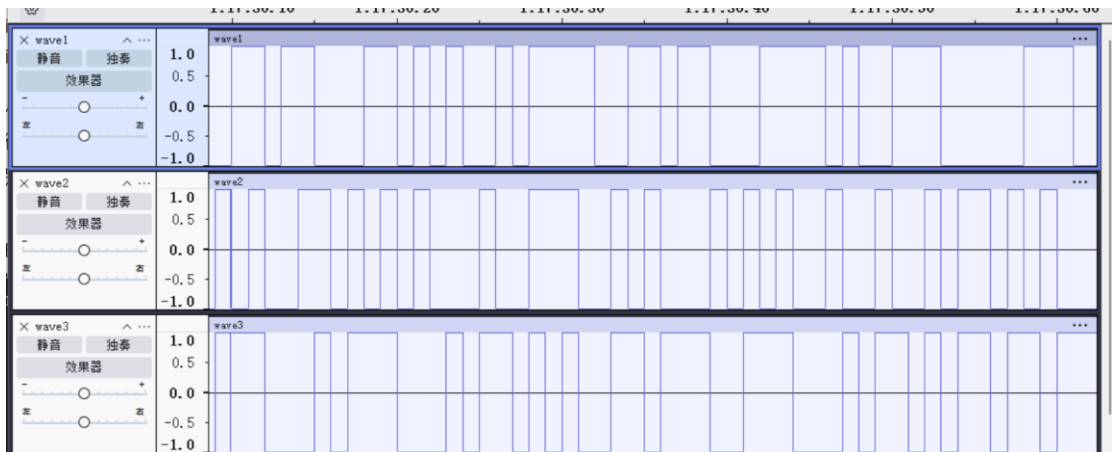
Misc+黑白有常，虹色寻异

解题思路（必须包含文字说明+截图）

1. 打开发现图片是一四位数 16 比特的颜色码，其他压缩包都需要密码，我们从这个图片下手
2. 图片是 RGB 图片，我们分别对三个通道进行二进制提取之后转换可能就能获得三个 wave 压缩包密码，我们先写脚本（ai 辅助）对每个颜色通道进行分别处理，提取最低为到最高位的所有位平面
得到三个密钥 I@mGr33N 对应 wave2 ReDDD_1z_G00d 对应 wave3 A_B1u3_K3YYY?!对应 wave1



3. 我们用 audacity 打开三段音频，发现波形都是存在 01 电平，我们分别提取 01 出来，每一段波就能组成一段二进制



4. 根据提示“你是否可以将听到的音频变为 406×228 的图片”，我们想到将三个音频转换位图片，， 每个音频对应 RGB 其中一个通道，



得到 key: sKJDh-

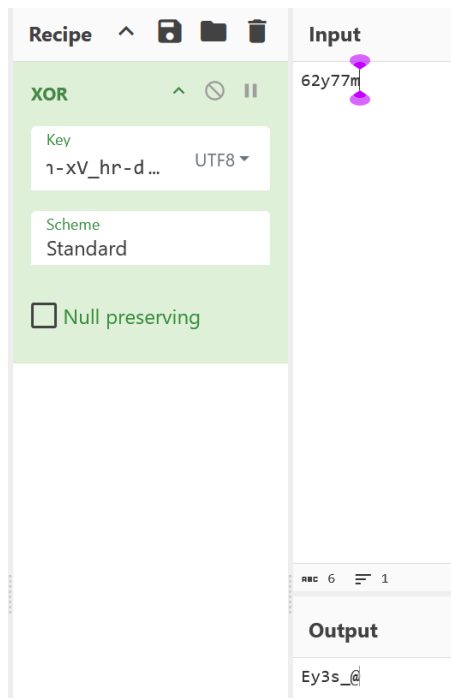
xV_hr-daX

5. 根据提示根据提示图片名 62y77m 可以和某一段 key 做运算，尝试异或运算
眼见为实？耳听为虚？你是否可以将听到的音频变为 406×228 的图片？七种颜色下埋藏的是怎样的秘密？阴与阳的结合又能迸发出怎样的火花？你能找到这其中的奥秘吗？

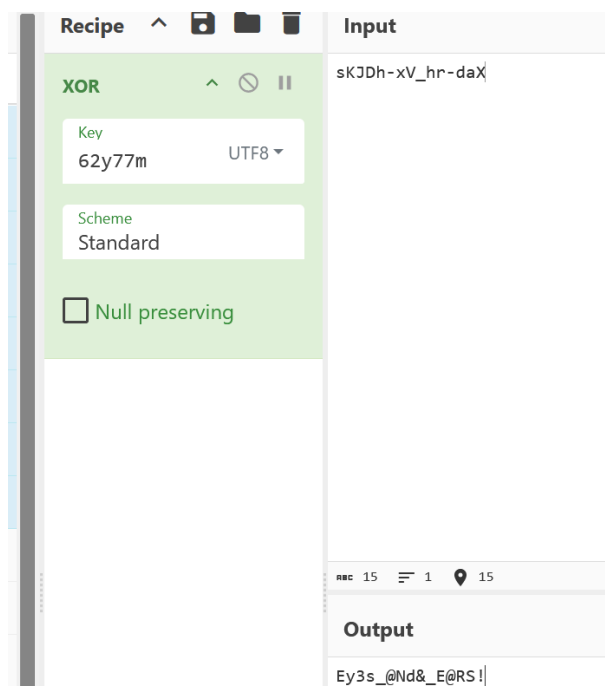
Hint 1: 计算机所展示出来的图片并非全貌，每一个通道不止8位

Hint 2: 最初的文件名可以和某一段key进行某种运算

Hint 3: 为什么最后一段压缩包要用rar压缩，在二进制中或许有信息



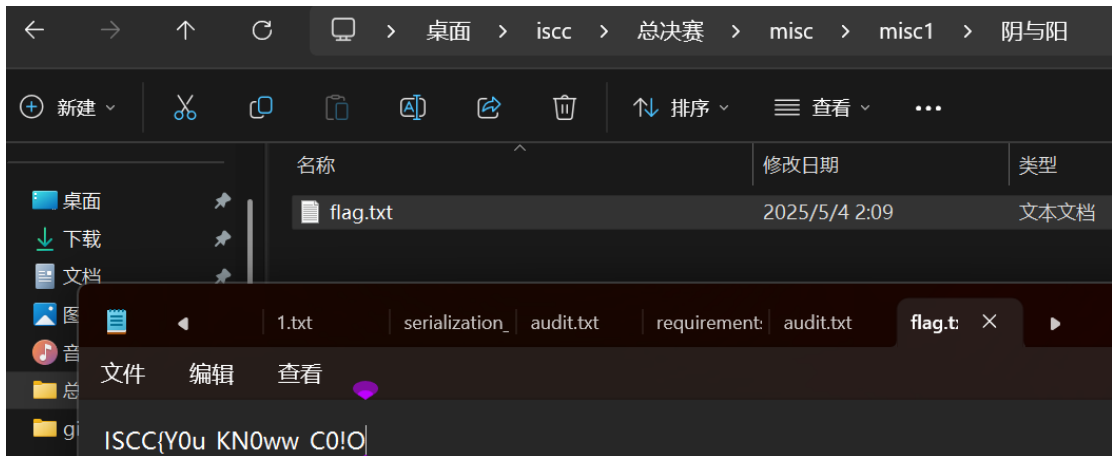
密码不对，顺序换一下



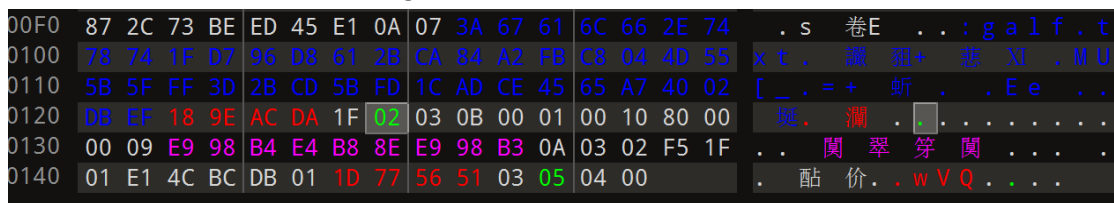
得到压缩包密码

Ey3s_@Nd&_E@RS!

6. 解压只得到一半的密码，根据提示压缩包二进制有答案



7. 可以看到里面还应该有一个 golf.txt, RAR 包含备用数据流 (ADS)

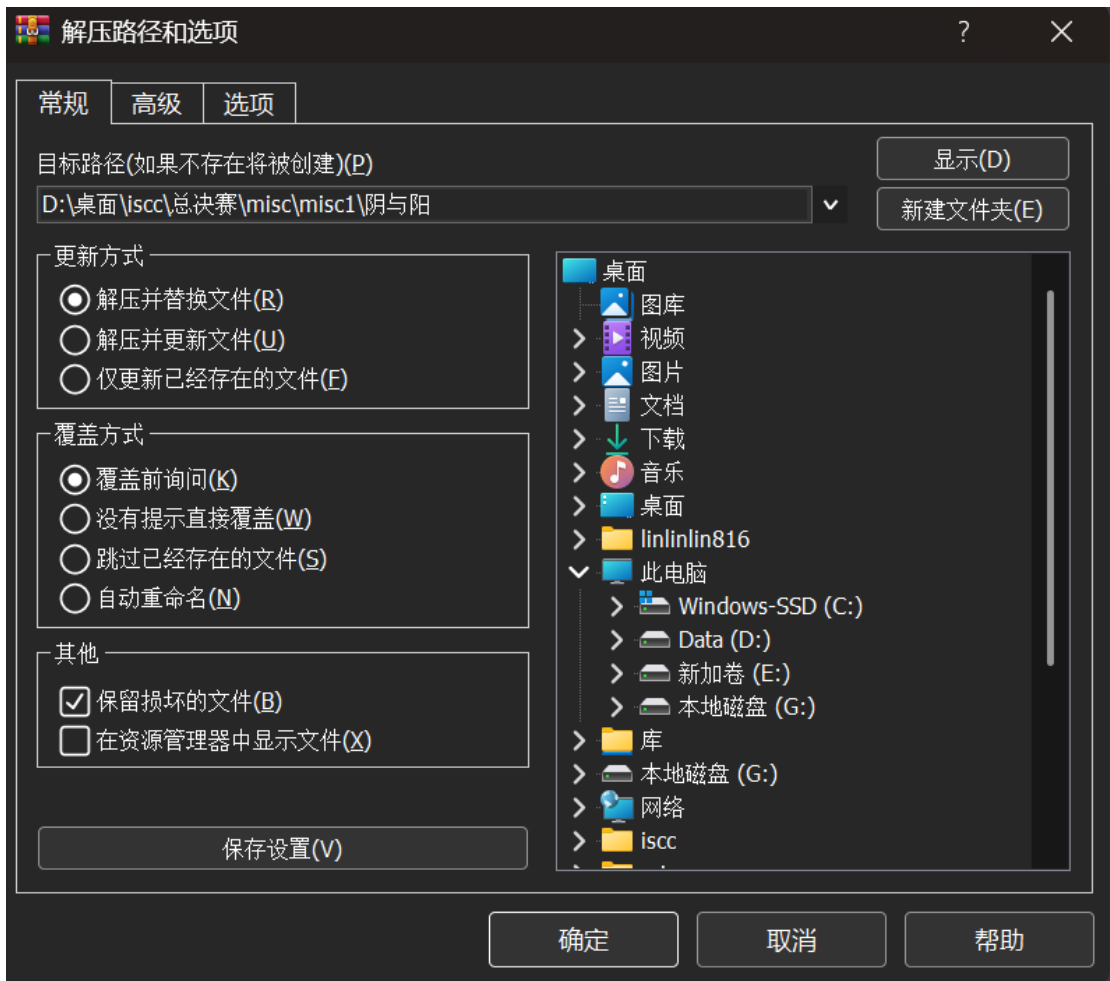


在 Window 环境下, 处理 NTFS 备用数据流的常用命令是 Get-Content -

Stream

```
PS D:\桌面\iscc\总决赛\misc\misc1\阴与阳> Get-Content .\flag.txt -Stream golf.txt
Get-Content : 无法打开文件“D:\桌面\iscc\总决赛\misc\misc1\阴与阳\flag.txt”的备用数据流“galf.txt”。
所在位置 行:1 字符: 1
+ Get-Content .\flag.txt -Stream galf.txt
~
+ CategoryInfo          : ObjectNotFound: (D:\桌面\iscc\总决赛\misc\misc1\阴与阳\flag.txt:String) [Get-Content], Fi
leNotFoundException
+ FullyQualifiedErrorId : GetContentReaderFileNotFound,Microsoft.PowerShell.Commands.GetContentCommand
```

8. 再看看题目提示为什么最后一段压缩包要用 rar 压缩。Emmm 是不是没有使用 winrar 解压导致数据丢失? 尝试一下, 记得勾选保留损坏文件



9. 提取成功，得到 ISCC{Y0u_KN0ww_C0!OuR_@nd_B1n@RY!!!}

```
PS D:\桌面\iscc\总决赛\misc\misc1\阴与阳> cd D:\桌面\iscc\总决赛\misc\misc1\阴与阳\阴与阳
PS D:\桌面\iscc\总决赛\misc\misc1\阴与阳\阴与阳> Get-Content .\flag.txt -Stream galf.txt
uR_@nd_B1n@RY!!!}
```

Exp（如有，请粘贴完整代码，不允许截图!）

```
import cv2
```

```
import numpy as np
```

```
import os
```

```
import wave
```

```
from PIL import Image
```

```
# 图像位平面可视化函数
```

```
def bit_planes(image_path):
```

```
    if not os.path.exists(image_path):
```

```

        raise FileNotFoundError(f"图像文件 {image_path} 不存在")

img = cv2.imread(image_path, cv2.IMREAD_UNCHANGED)
if img.ndim > 2:
    img = img[:, :, :3]
else:
    img = np.stack([img] * 3, axis=-1)

h, w = img.shape[:2]
max_bit = 16 if img.dtype == np.uint16 else 8
grid = np.zeros((h * 3, w * max_bit), dtype=np.uint8)

for c in range(3):
    channel = img[:, :, c]
    for bit in range(max_bit):
        bit_plane = ((channel >> bit) & 1) * 255
        grid[h * c:h * (c + 1), w * bit:w * (bit + 1)] = bit_plane

output_path = "bit_planes_result.png"
cv2.imwrite(output_path, grid)
print(f"位平面结果保存到: {output_path}")
return output_path

```

音频转图像相关函数

```

def extract_01_sequence(wav_path, interval=0.01):
    with wave.open(wav_path, 'rb') as wf:
        samples = np.frombuffer(wf.readframes(wf.getnframes()),
                                dtype=np.int16 if wf.getsampwidth() == 2
                                else np.uint8)
        samples_per_interval = int(wf.getframerate() * interval)

```

```

return ''.join('1' if samples[i * samples_per_interval] > 0 else '0'
               for i in range(len(samples) // samples_per_interval))

```

```

def bits_to_channel(bits, pixel_count):
    return np.array([int(bits[i:i + 8], 2) for i in range(0, min(len(bits), pixel_count *
8), 8)],
                    dtype=np.uint8)

```

```

def audio_to_image():
    width, height = 406, 228
    pixel_count = width * height
    channels = {wav: bits_to_channel(extract_01_sequence(wav), pixel_count)
               for wav in ['wave1.wav', 'wave2.wav', 'wave3.wav']}

    max_len = max(len(ch) for ch in channels.values())
    for wav in channels:
        if len(channels[wav]) < max_len:
            channels[wav] = np.pad(channels[wav], (0, max_len -
len(channels[wav])))

```

```

img_data = np.stack([channels[f'wave{i}.wav'] for i in range(1, 4)], axis=1)
img_data = img_data.reshape((height, width, 3))

```

```

img = Image.fromarray(img_data, 'RGB')
output_path = "audio_to_image_result.png"
img.save(output_path)
print(f"音频转图像结果保存到: {output_path}")

```

```
if __name__ == '__main__':  
    try:  
        # 示例：处理图像（需存在 62y77m.png）  
        bit_planes("62y77m.png")  
        # 示例：处理音频（需存在 wave1-3.wav）  
        audio_to_image()  
    except Exception as e:  
        print(f"错误: {e}")
```